

Lecture Note: Comparison, Logical Operators & Conditional Statements

Nishut Suman

5 Jan, 2026 At 9:00 PM

Notes

Foundation

Recommended

Resource



Associated Lectures (1)



Description



Discussions



Lecture Notes: Comparison, Logical Operators & Conditional Statements

What you'll be able to do:

- Apply comparison and logical operators to solve real-world problems in Python
- Write multi-level conditional statements to make smart programmatic decisions
- Identify and avoid common pitfalls in Python decision logic

1. Why Comparison, Logical Operators & Conditionals Matter

Python programs often need to **make decisions**. From deciding if a user can log in, to checking if a number is prime, **conditions control the flow** of your program. Mastering these enables you to write intelligent, responsive code.

Real-world example:

- E-commerce websites use conditions to apply discounts only if a user meets certain criteria (e.g., membership + minimum cart value).
- Traffic lights use logical conditions internally: *if sensor detects cars AND light is red → switch to green.*

2. Advanced Comparison Operators Concepts

Beyond `==`, `!=`, `<`, `>`, `<=`, `>=` Python supports **chained comparisons** and **identity checks**:

Chained Comparisons

```
x = 10
print(5 < x < 15)  # True → checks both 5 < x AND x < 15
```

Tip: Chaining improves readability and reduces errors from multiple **and** conditions.

Identity and Membership Operators

- **is** → checks if two objects are the same in memory
- **is not** → negation of **is**
- **in** → checks if an element exists in a collection
- **not in** → negation of **in**

```
lst = [1, 2, 3]
a = 2
print(a in lst)  # True
print(a is 2)   # True, but careful with larger objects
```

Common confusion:

```
== checks value equality
is checks object identity. Beginners often mix these up.
```

Reference diagram:

- [Comparison vs Identity Operators](#)

3. Logical Operators Deep Dive

Logical operators can be combined **to form complex conditions**:

```
age = 25
has_ticket = True
is_member = False

# Enter only if age > 18 AND has_ticket OR is a member
if (age > 18 and has_ticket) or is_member:
    print("Access granted")
```

Key points:

- **and** → both must be True
- **or** → at least one must be True
- **not** → flips the boolean value

Pitfall: Misunderstanding precedence. Use parentheses to clarify logic.

Reference visual:

- [Python Logical Operators](#)

4. Nested & Complex Conditional Statements

Python allows **nested and multi-branch decisions**:

```
score = 78

if score >= 90:
    grade = "A"
elif score >= 70:
    grade = "B"
    if score > 75:
        print("Good effort!")
else:
    grade = "C"

print(f"Grade: {grade}")
```

Real-world analogy:

- A university assigns grades based on score ranges, then gives additional honors based on specific achievements.

Tip: Avoid deep nesting; extract logic into functions when complexity grows.

Reference diagram:

- [Conditional Ladder Flowchart](#)

6. Common Beginner Pitfalls & How to Avoid Them

Mistake	Why It's a Problem	Correct Approach
Using = instead of ==	Assignment instead of comparison causes syntax errors	if x == 5:
Over-nesting conditions	Hard to read and maintain	Break logic into functions or use elif chains
Ignoring logical precedence	Can lead to unexpected results	Use parentheses *(a and b) or c
Comparing different types	"5" > 3 → error	Convert types explicitly: int("5") > 3

Mistake	Why It's a Problem	Correct Approach
Misusing is vs ==	Confusing value equality with identity	== for values, is for objects

7. Practice Exercises

Basic Conditional: Check if a number is positive, negative, or zero.

```
# Hint: use if, elif, else
```

Logical Operators Challenge: A theme park allows entry if the person is either a VIP or has a ticket **and** is above 18.

```
# Hint: combine 'and' and 'or' with parentheses
```

Nested Condition Challenge: Classify exam scores into:

- = 90 → Excellent
- 70-89 → Good
- 50-69 → Average
- <50 → Fail

Extension: Add special messages for borderline cases (e.g., 69 or 89).

8. Key Takeaways

- **Comparison operators** → check relationships between values
- **Logical operators** → combine multiple conditions efficiently
- **Conditionals** `if, elif, else` → control program flow
- **Nested conditions** → improve logic and performance
- Always **use parentheses for clarity** in complex expressions

Mental Model: Think of Python decision-making as a flowchart in your mind. Each condition directs the flow, guiding your program’s “choices.”



Additional References

- [Python Docs: Comparisons](#)
- [Python Docs: Boolean Operations](#)

